LGBT Center Library Catalog

Boris Brondz, Daniel Burwell, Hannah Hejazi, Deborah Park, Natalie Qiu, Fei Triolo

Source Code: https://github.com/bxb454/csds-395-lgbt-library-catalog

Problem Statement

The CWRU LGBT Center hosts a small library containing 4 shelves of books, DVDs, and other borrowable documents. The current Library Management System is a Google Sheet containing most of the existing catalog, and a Google Form for patrons to check out and return books. This current system has significant overhead for both patrons who struggle to find and checkout relevant materials, as well as LGBT Center employees who are required to meticulously update the spreadsheet and personally track patrons' due dates.

Specifications / Requirements

We intended to develop a full-stack web application and database that provides a searchable interface for patrons to browse relevant material, checkout, renew, and track their due dates. The application should also contain accessible Library Management features for LGBT Center Staff to update, add and remove items without having technical knowledge of the backend. We used a server running Apache httpd provided by UTech to host the application on dedicated server space with a public-facing website for future LGBT Center staff and visitors to use.

The application interface design should be easy for the average reasonable person to use, secure to protect the privacy of patrons, and appealing to look at. The database structure should be robust enough to need minimal maintenance in the future to minimize the burden on the LGBT Center staff. We will nonetheless provide documentation for future updates.

Background & Related Work

Fei has worked on similar Library Management backend projects for courses such as CSDS293—Java-based Library Management software architecture, and CSDS341—MySQL database and rudimentary JDBC front end.

We are drawing inspiration from existing public library catalogs such as the Heights Libraries (https://heightslibrary.org), the Cuyahoga County Public Library (https://cuyahogalibrary.org/), the King County Library System (https://kcls.org/), and the Cleveland Public Library (https://cpl.org).

For the backend and database creation, we are using Go and MySQL, and React.js for the front-end interface. Additionally, we may experiment with a Spring Boot back-end to prioritize security.

Finally, we will be utilizing the university's CAS Single Sign-On system in order to manage logging into and out of the library catalog, documentation for which can be found at https://case.edu/utech/help/knowledge-base/cwru-network-id-passphrase/cas-single-sign-cwru-kba.

Pre-existing database schema:

As stated above, the current method used to keep track of the library catalog is a spreadsheet which can be browsed by potential clients and must be manually updated by staff members, which can be found at https://docs.google.com/spreadsheets/d/1tbhRvgI_Dn5JIICZVFxTj26xtTF0JUEa-iRT5JDvmvs/htmlview. Patrons can check out books by filling out the Google form linked in the spreadsheet, at https://forms.gle/pk3VKTN5Bux59rTp9.

Completed Work

Team Contributions

This project was divided into 4 teams:

- Database Configuration: Fei, Hannah
- Frontend: Deborah, Natalie
- Backend: Boris, Daniel
- Server Administration: Natalie, Fei

The role of the frontend team was to create the UI of the website using React and TypeScript via Vite. The backend team was responsible for managing the creation of the database, as well as managing database queries sent from the frontend. Additionally, they were responsible for integration with the CAS system for handling users and authentication.

Fei worked on the database schema and set up prepared statements, and filmed the demonstration video. Boris and Dan worked on the Go backend. Boris created the framework for the API server and worked on pagination and the API endpoints for handling books (including individual books by using the book's ID), users, and searching. Dan built the API endpoints for handling loans, including renewing and returning (i.e. individual loans by ID), and authors. Natalie worked briefly on the front-end prototype, but Deborah implemented the final version, including

incorporating axios calls to connect it to the backend. Hannah worked on project reports and presentations, configuring the server, and entering data into the database. Natalie spoke with UTech to obtain server space and worked on configuring the server, along with Fei.

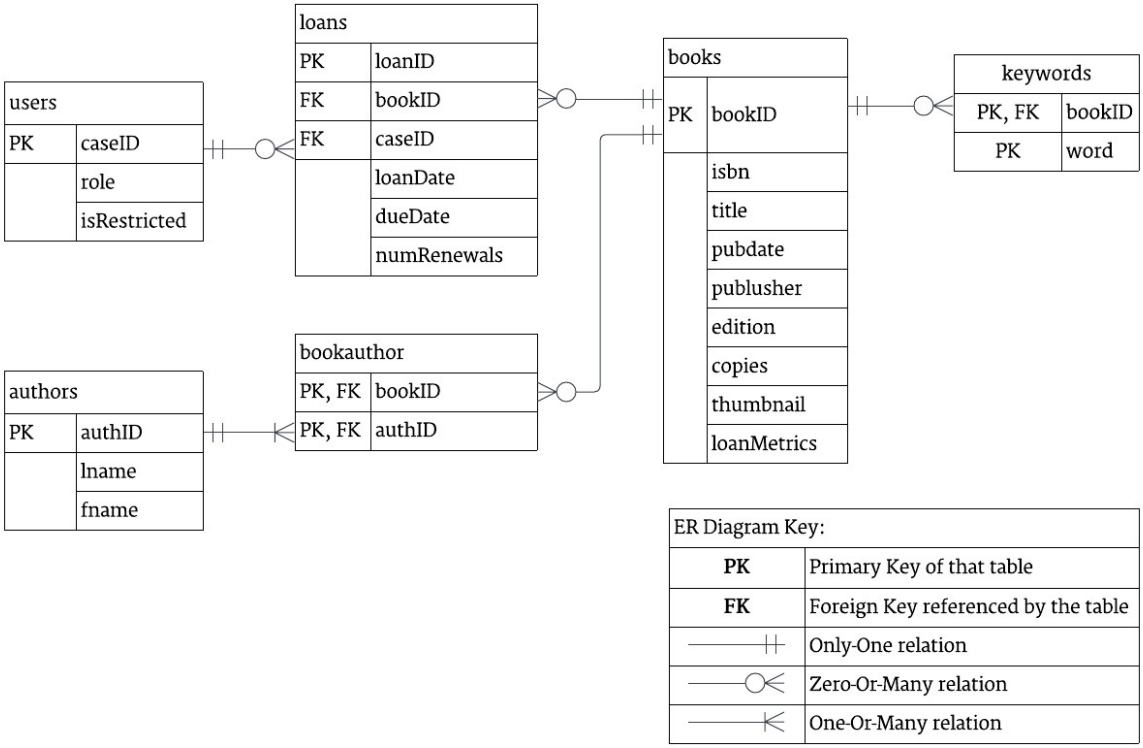
Relational Database Schema Development

Based on previous project experience, the database schema was carefully designed to maximize a number of considerations, including space efficiency, backend integration, and design requirements.

Our catalog must be easily searchable, both by intrinsic properties of a book (i.e. title, publisher information, authors), and by metadata such as genre, subject matter, medium, etc. We decided to abstract this metadata into a keyword system in which each book can be related to multiple user-defined keywords. This allows flexibility for the organization to determine what relevant metadata is important for them to associate with each book. Additionally, we decided to store author data separately from individual books, related to each other by the bookauthor transition table. This enables a many-to-many relation between books and authors, which minimally accounts for cases where a book has multiple authors, each equally searchable, or where the same author contributes to multiple books in the collection.

Our primary concern for the user validation and loan tracking is user privacy. Due to social stigma associated with the LGBT community, patrons may have aversions to the possibility of their patronage of center resources being traced back to their personal identities. Additionally, many patrons may otherwise be uncomfortable seeing the same name associated with their University account used in the context of LGBT community spaces (such as a transgender student who is privately out amongst regular LGBT Center attendants but still wishes to remain closetted in their larger academic life). Ultimately, we decided to store as little user information as possible without sacrificing critical functional requirements (i.e. caseID, user role, and loan restriction status). This rationale also informed our decision to make loan information temporal—that is, when a loan is returned, we discard all receipts of that loan associated with that user.

Below is the full Entity Relation (ER) Diagram for our database schema:



While the schema itself was relatively straightforward to design and implement in a MySQL instance, we faced some technical challenges in the development workflow. While we were waiting to get server space from the university, we had nowhere to host our development instance. Since GitHub is not optimal for hosting relational databases, the group members who focused on database development opted to locally host their own instances of the database in their machines. This presented difficulties in version control, having to manually update the schema across all active instances whenever new database definition queries were written.

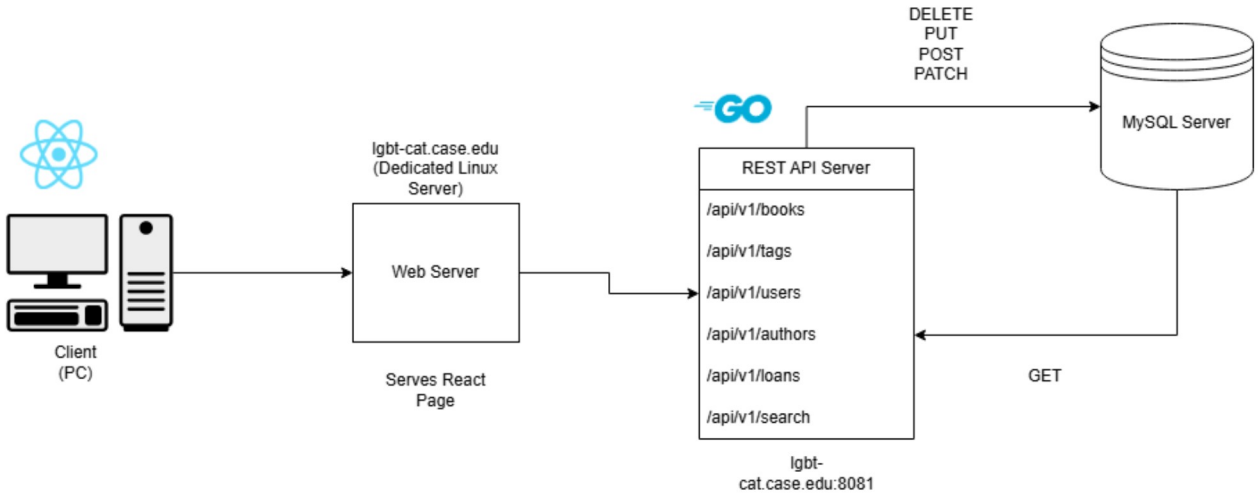
Backend API Development

To create a layer of abstraction between the frontend and database, we devised a Golang-based backend server to facilitate all communication between the frontend and database. Aside from being a standard procedure when developing web applications, the creation of a REST API allows for:

1. Secure transactions (prepared SQL queries that are parameterized - no need to worry about basic attacks such as SQL injection).
2. Limit concurrent access to the application to prevent denial of service attacks (through the introduction of rate limiting)

3. Effectively tackle the issue of scalability. The backend can be scaled independently of the database and frontend to accommodate traffic relative to other components.
4. Utilize the API as a standalone application that can be utilized as a service for anyone else interested in / intending to work with the LGBT center on anything that utilizes data in the LGBT Library Catalog.

System Design Diagram for the current iteration of the LGBT Library Catalog App:



The reasoning behind using Golang as opposed to a language like Java or JavaScript or even Python relates to the concept and purpose of Go as a programming language. Go is much more lightweight and versatile, and supports cross-compilation with different CPU architectures (x86, x64, ARM, etc..). The ease of use of Go for backend development allows minimal overhead in terms of packages and libraries as well. In addition, Go offers native support for building files into runnable executables, and has a robust standard testing library. Golang is a language that also “forces” proper programming practices onto its developers; for example, throwing compiler errors for unchecked errors, unused fields/libraries, etc.

Role Members Assigned: Boris Brondz, Daniel Burwell

Boris Responsibilities:

1. Build API endpoints for books, search, and users
2. Incorporate pagination into handlers + rate limiting
3. Write documentation for the backend server for the frontend team

4. Develop integration tests in an isolated environment (Testcontainers) to test communication between the REST API and the MySQL DB

Daniel Responsibilities:

1. Build API endpoints for loans and authors
2. Handle code for checking out, renewing, and returning books
3. CAS Authentication for third-party CWRU Single Sign-on (SSO) integration

NOTE: All documentation for the backend API is located in our GitHub repository in the backend folder. The documentation is a README file containing instructions to set up and run the backend server on a local machine. The README also contains sample API endpoints to help the frontend team understand how they can implement calls to the backend via fetch, axios libraries (examples for both provided) + cURL for simple testing. There is also a new and improved README file specifically for axios calls - this is for the frontend team (for integration help with the frontend).

In terms of core functionality, the REST API server's core functionality is essentially complete. The main priorities for future iterations of development for the REST API are laid out in the list of future plans below.

Backend REST API - Development

Dedicated API Base URL: https://lgbt-cat.ugen.case.edu:8081/api/v1/

The REST API for the LGBT Library Catalog supports 6 data-based endpoints:

/api/v1/books/ -> Supports GET, PUT, POST, DELETE operations
/api/v1/loans/ -> Supports GET, PUT, POST, PATCH, DELETE operations
/api/v1/users/ -> Supports GET, PUT, POST, DELETE operations
/api/v1/search/ -> Uses query header (ex: /api/v1/search?q=param1&limit=10&offset=0)
/api/v1/tags/ -> Supports GET operations on that endpoint alone but books handles the "joint" operations (/books/{bookID}/tags/ etc...)
/api/v1/authors/ -> Supports GET, POST operations

Each endpoint has a prefix associated with "v1" of the api - or its intended first iteration. We are using API versioning as it is a general industry-standard practice to "version" APIs; future iterations could use a "v2" as a version, for example (as an upgrade over the first iteration).

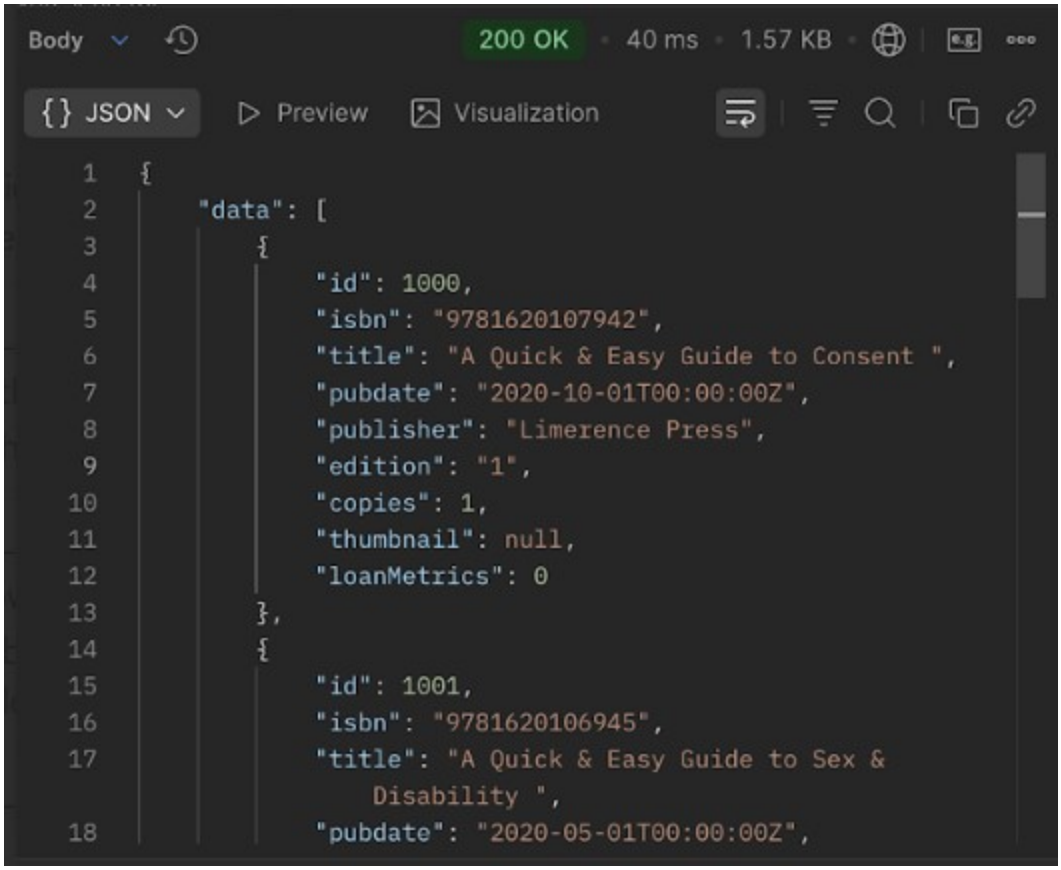
Each endpoint (including Daniel's endpoints /authors/ and /loans) is designed to be a handler function, in which they return anonymous functions that are designed to handle separate HTTP REST operations (through switch statements) based on the type of incoming request (GET, PUT, POST, PATCH, DELETE) and these anonymous functions will handle logic accordingly. In addition, each endpoint is paginated.

```
v1 := http.NewServeMux()
//boris endpoints
v1.Handle("/books", s.WrapLimiter(s.HandleBooks()))
//note: the trailing slash is important here to match /books/{id}
v1.Handle("/books/", s.WrapLimiter(s.HandleBookByID()))
//extra endpoint for getting book-author relationships (totally forgot about this)
//NOTE: ONE ENDPOINT PER FUNCTION OR ELSE THERE WILL BE CONFLICTS
//v1.Handle("/books/", s.wrapLimiter(s.handleBookRelations()))
v1.Handle("/search", s.WrapLimiter(s.HandleSearch()))
v1.Handle("/users", s.WrapLimiter(s.HandleUsers()))
//same here
v1.Handle("/users/", s.WrapLimiter(s.HandleUsers()))
v1.Handle("/tags", s.WrapLimiter(s.HandleTags()))
//endpoints made by dan:
v1.Handle("/authors", s.WrapLimiter(s.HandleAuthors()))
v1.Handle("/loans", s.WrapLimiter(s.HandleLoans()))
// once again, trailing '/' is important here
// url extension will be in the form "/loans/{loanID}, or /loans/{loanID}/renew"
v1.Handle("/loans/", s.WrapLimiter(s.HandleLoans()))
```

Each endpoint is handled under the same request "router" to ensure that each endpoint is served by a singular function, under one connection, with each endpoint being its own separate HTTP handler function supporting all necessary REST operations. This is currently a monolithic setup, and future plans include having each endpoint as its own independently-launched microservice.

Sample request (GET http://lgbt-cat.ugen.case.edu:8081/api/v1/books/):

This sample request retrieves all books from our MySQL database.

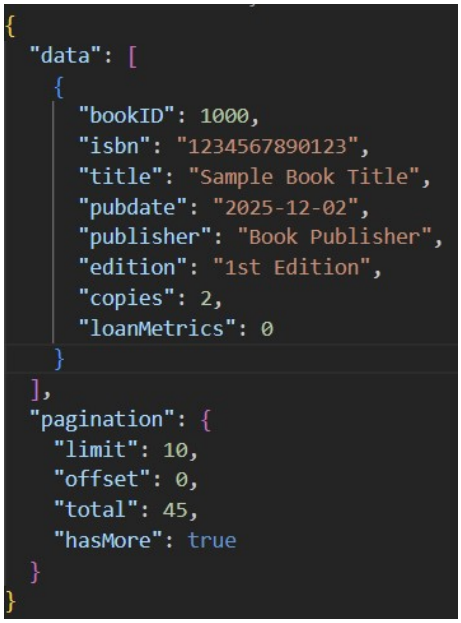


Rate Limiting:

```
s := &Server{
    Db:           db,
    router:       http.NewServeMux(),
    limiters:     make(map[string]*rate.Limiter),
    rateInterval: time.Second / 10,
    rateBurst:    10,
}
```

As mentioned earlier, the API server implements rate handling. In this case, our setup is currently set to handle 10 requests per second (1 second / 10 meaning) at the very maximum (returning a 429 Too Many Requests error if we exceed that limit at any given point), and we are set to handle 10 concurrent requests at any given point (rateBurst).

Pagination:



For the sake of brevity - other books in this dummy call are omitted.

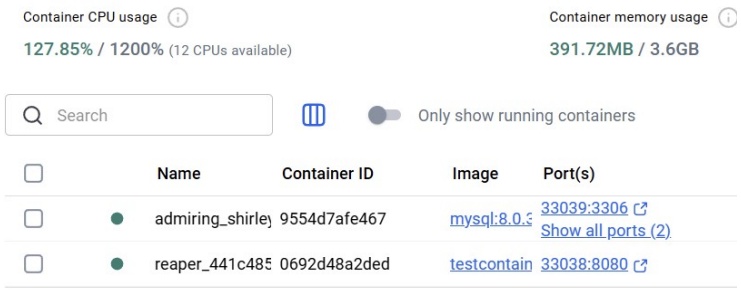
In this case, limit defines the total number of objects to return per GET request. In this case, 10 books. Offset defines where we start; for instance, if we define offset as 10, this means we start from the 10th book onwards. Total defines how many total objects (in this case books) are present in the MySQL database. hasMore is a boolean that determines whether there are more “pages” to go through (determined by the formula: offset + limit < total).

Integration Testing (Backend)

Integration tests were implemented as a means to test proper communication between the REST API and the MySQL database. In order to prevent additional overhead on our dedicated server, we opted for integration testing in an isolated local environment, made possible with the Testcontainers framework. Testcontainers makes it possible to create temporary, lightweight containers that are automatically loaded and offloaded once execution of tests is complete.

Since the order of execution of tests in Golang is randomized (a practice enforced to ensure test logic remains independent of other tests), a test “driver” function (TestMain) was required to setup the MySQL container (with proper DDL and DML scripts for setting up the proper database environment and inserting fake dummy values into it).

The test functions test all endpoints with their own supported REST operations, but tests for PATCH operations (supported by /authors/ and /loans/) are yet to be implemented. In fact, PATCH operations might also be introduced for the other



This is a screenshot from Docker Desktop on my local system, showing Docker Engine tracking the temporary containers made by Testcontainers while integration tests were running.

```
--- PASS: TestLoansGet (0.04s)
=== RUN   TestLoansLifecycle
=== RUN   TestLoansLifecycle/Create
    loans_integ_test.go:104: Created loan with ID: 1003
=== RUN   TestLoansLifecycle/Delete
    loans_integ_test.go:154: Successfully deleted loan ID: 1003
--- PASS: TestLoansLifecycle (0.15s)
    --- PASS: TestLoansLifecycle/Create (0.05s)
    --- PASS: TestLoansLifecycle/Delete (0.09s)
=== RUN   TestSearch
--- PASS: TestSearch (0.09s)
=== RUN   TestTagsGet
--- PASS: TestTagsGet (0.04s)
=== RUN   TestUsersGet
--- PASS: TestUsersGet (0.05s)
=== RUN   TestUsersPost
--- PASS: TestUsersPost (0.05s)
=== RUN   TestUserGetSingle
--- PASS: TestUserGetSingle (0.04s)
=== RUN   TestUserDelete
--- PASS: TestUserDelete (0.05s)
PASS
2025/12/05 02:43:51 🐳 Stopping container: 3baaa1a6cc3c
2025/12/05 02:43:53 ✅ Container stopped: 3baaa1a6cc3c
2025/12/05 02:43:53 🐳 Terminating container: 3baaa1a6cc3c
```

This is a snippet of test execution, showing passing tests (all tests pass indicated by the final PASS line - if one test (or multiple or all) failed it would be “FAIL”). Console output also shows that the containers are offloaded as soon as tests are complete.

CAS Authentication (Backend)

While there is a button on the server connected to the SSO link to login, CAS Authentication has not been implemented yet due to authorization issues, and will be investigated more closely as the application is developed.

Full Stack Frontend Design

Our web-based frontend uses Vite and React. Currently, the tabular views are built with material-ui’s react-table component, which supports CRUD operations and pagination. The website allows for navigation through the catalog pages and the ability to view a certain number of items at a time. A modal window appears when a user selects an item that includes more details about the item. The form for staff to add new titles to the catalog is also simple and intuitive to use, stating what fields are required and providing information to the user about incorrect inputs that result in a title not being added. The search bar for browsing the catalog also allows the user to search by attributes such as title, author, ISBN, keyword, or date of publication before/after a certain date. Users are also only exposed to page navigation/actions specific to their given role (patron, staff, admin). The current design was created with ease of use in mind, but we anticipate making tweaks based on future tester feedback.

Dedicated Server Setup

We obtained the VM lgbt-cat.case.edu from uTech, with the 6 of us having sudo access.

Currently, the server is only accessible through the CWRU campus network. Ideally we would later have it accessible to the world, but that requires us to implement a level of security that we can’t currently achieve.

The server is running Apache httpd in a default configuration, with both SSL (HTTPS / port 443) and insecure (HTTP, port 80) enabled. The server currently uses a default self-signed certificate. Assuming the project is successful and goes into production at the LGBT center, we can obtain a valid SSL certificate through utech.

The frontend is served from a copy of its static built files located in the default "webroot" directory, /var/www/html/ . The API and database are running as systemd services. The

db is accessible on a port that the api calls through a password-protected DSN and the the API is running out of port 8081.

Configuration Files and Deployment Scripts

The server has too many configuration files. We only utilized some of them,
The webend configuration is located at /etc/httpd/conf/httpd.conf. CAS login has its own configuration file at /etc/httpd/conf.d/auth_cas.conf.
We wrote a bash script to automatically deploy the front end, which is located in the root directory. The script allows us to pick which branch to deploy, allowing us to test versions directly on the server that aren't in the main branch. The main branch was supposed to keep the last functional working prototype.

Cybersecurity Concerns

We have a few concerns in our implementations.

1. The API currently doesn't require a key. This is terribly insecure, considering we have full CRUD operations through the API, which allows anyone who knows the end point to create, delete, and update whatever entries they want.
2. Case IDs are currently stored as plaintext. We should hash these instead to fully protect user privacy, though this will inevitably require additional steps for authenticating checkouts.
3. There exists glitches in which logging out does not kick you out from viewing users or other Admin privileged information. We might want to restructure our page directories in order to fully block these sections behind CAS.

Did we meet our design specifications?

While the current state of our project is not where we would have liked it to be, there are some points which have been successfully implemented. We have a functional website connected to a university-hosted server that allows for users to search for books within our database, check them out, and view their loans. There is also support for add/remove operations. We have plans for further development that will enable our web service to be a useful resource for future staff and patrons of the LGBT Center.

Future Plans:

1. Work on a comprehensive logging system (possibly a centralized logging system) to handle and manage messages/errors produced by REST API calls on the server.

2. Creating additional unit tests by creating “mock” implementations of our REST handler functions to simulate “mock” API calls - tests at the smallest level to test for any particular edge cases.

3. Re-modularize backend code to reduce boilerplate and better follow Golang syntax conventions. This still stands as the code still contains a lot of boilerplate which can be further collapsed and reduced into various helper functions, and we can think of using API development libraries like gorm, gin/gonic, gorilla, etc..

4. Split the monolithic back-end into microservices, with each endpoint being its own standalone service that can scale separately. We can utilize API Gateway architecture to route and handle requests for every endpoint. We can containerize the files for each separate endpoint and create a docker-compose YAML file so that we can instantaneously launch every microservice at once.

5. Further improve integration test coverage by adding additional tests for PATCH, extra DELETE endpoints, etc, and any other missing endpoints not covered by current iteration of tests, and to introduce unit tests. The goal here is to improve code coverage.

6. Develop a Grafana dashboard that tracks API performance, downtime, uptime, total operations performed, total amount of books in DB, etc. by integrating Prometheus metrics within the code for each endpoint. This dashboard will be used for internal purposes only (a Prometheus metrics endpoint as a data source).

7. A fully working CI/CD pipeline (Jenkins/GitHub Actions/etc...) that automates the entire build + testing + deployment process (to the dedicated server space) to ensure absolute minimum downtime between code updates (a flow that can be triggered when there is a commit/merge to main).

8. Add additional supplemental features, such as integration of emailing services, for the purpose of sending confirmations to patrons and staff after successfully checking out, or sending out reminder emails for upcoming due dates. Additionally, a suggestion form could be created for suggesting books to add to the library, which could send an email to staff.

9. UX tweaks and accessibility. We hope to improve the UI flow and API logic through beta testing and iterative design. Additionally, improving the frontend compatibility across devices, browsers, and themes, adding screen reader compatibility, alt text, and other accessibility features.

Conclusion

Despite the entire semester's worth of work we've put into this project, there is still much that must be done next semester. Those of us who are not graduating are still interested in working on the LGBT catalog, and we hope to have it deployable in the next semester.